

System Data Cleaner for macOS

Complete Product Requirements and Engineering Specification

Project Type: Native macOS Utility

Primary Purpose: Analyze large storage-consuming files and directories and provide a safe graphical cleanup interface

Platform: macOS

Language: Swift

UI: SwiftUI

Core Frameworks: Foundation + macOS system APIs

Architecture: Lightweight MVVM + service-based architecture

Network: Not required

Backend: None

Database: None required for MVP

AI/ML: None

Background Service: None by default

Distribution: Direct distribution / notarized DMG initially

1. Project Overview

1.1 Project Name

Working name:

System Data Cleaner

Possible final names:

- SystemClean
- SpaceClean
- System Data Cleaner
- SpaceLens
- Mac Space Cleaner

The final branding can be selected later.

2. Executive Summary

macOS users can inspect storage through System Settings, but macOS does not provide a sufficiently detailed and convenient interface for investigating everything that may contribute to large amounts of "System Data."

A user may see:

System Data
143 GB

but may not know what is responsible for this storage consumption.

Users who understand Terminal can manually investigate storage using commands and utilities that enumerate directories, calculate file sizes, search for large files, inspect caches, and remove unnecessary data.

However, these operations require:

- Terminal knowledge
- Knowledge of filesystem paths
- Knowledge of Unix commands
- Understanding of macOS permissions
- Care when deleting files

The proposed application converts this workflow into a **small native graphical utility**.

The application will:

1. Scan accessible storage locations.
2. Calculate the sizes of files and directories.
3. Identify large storage consumers.
4. Organize results into understandable categories.
5. Allow users to inspect individual items.
6. Mark potentially safe cleanup candidates.
7. Allow users to select unwanted items.
8. Validate selections.
9. Move appropriate items to Trash or delete them where appropriate.
10. Show the result after cleanup.

The application is intentionally focused.

It is **not intended to become a general-purpose "Mac optimizer."**

3. Core Product Idea

The fundamental idea is:

Replace complicated Terminal-based storage investigation with a simple graphical interface.

Conceptually:

Traditional approach

```
Terminal
↓
Filesystem commands
↓
Text output
↓
User interprets paths
↓
User decides what to delete
↓
Terminal deletion command
```

System Data Cleaner

```
GUI
↓
Storage Scanner
↓
Visual results
↓
User inspects items
↓
User selects items
↓
Safety validation
↓
Trash / deletion
```

4. Product Philosophy

The application follows five principles:

4.1 Analyze First

Never start by deleting files.

4.2 Show the Actual Data

Users should see:

- Path
- Name
- Size
- Type/category
- Modification date where useful
- Safety information

4.3 User Makes the Decision

The application should recommend but should not silently delete files.

4.4 Safety Before Cleanup

A large file is not necessarily an unnecessary file.

4.5 Stay Lightweight

The application should itself consume very little:

- disk space
 - RAM
 - CPU
 - battery
 - network bandwidth
-

5. Problem Statement

The primary problem is:

Users cannot easily determine which files and directories are responsible for large amounts of System Data on macOS or safely clean unnecessary data without using Terminal commands or third-party tools.

Example:

macOS Storage

System Data
150 GB

The user wants to know:

What is the 150 GB?

The application should turn that into:

System Data
150 GB

Docker	28.4 GB
Xcode DerivedData	21.7 GB

Application Caches	16.2 GB
iOS Simulator Data	14.8 GB
Logs	7.4 GB
Application Support	6.9 GB
Other	54.6 GB

6. Goals

Primary Goals

The application must:

- Analyze storage.
- Find large files.
- Find large directories.
- Present results graphically.
- Provide filesystem paths.
- Allow sorting.
- Allow searching.
- Allow selecting items.
- Explain selected items.
- Provide safety information.
- Allow cleanup.
- Verify cleanup results.
- Work without an internet connection.

Engineering Goals

The application should:

- Be native.
- Be lightweight.
- Be responsive.
- Be privacy-preserving.
- Avoid unnecessary dependencies.
- Avoid background processes.
- Avoid unnecessary privileges.
- Use macOS APIs wherever possible.

7. Non-Goals

The application will not initially provide:

- Antivirus functionality
- RAM cleaning

- CPU optimization
- Registry cleaning
- Startup optimization
- VPN functionality
- Internet optimization
- Cloud storage
- AI analysis
- Account/login system
- Online dashboard
- Continuous filesystem monitoring
- Automatic cleanup
- Generic system modification
- Kernel extensions

These are outside the project's core purpose.

8. Target Users

8.1 Normal Mac Users

Users who want to understand why their storage is full.

8.2 Developers

Users with:

- Xcode
- Docker
- Node.js
- Python
- Homebrew
- Android Studio
- virtual machines

8.3 Students

Users who frequently install development environments and large software packages.

8.4 Power Users

Users who currently use Terminal commands to investigate storage.

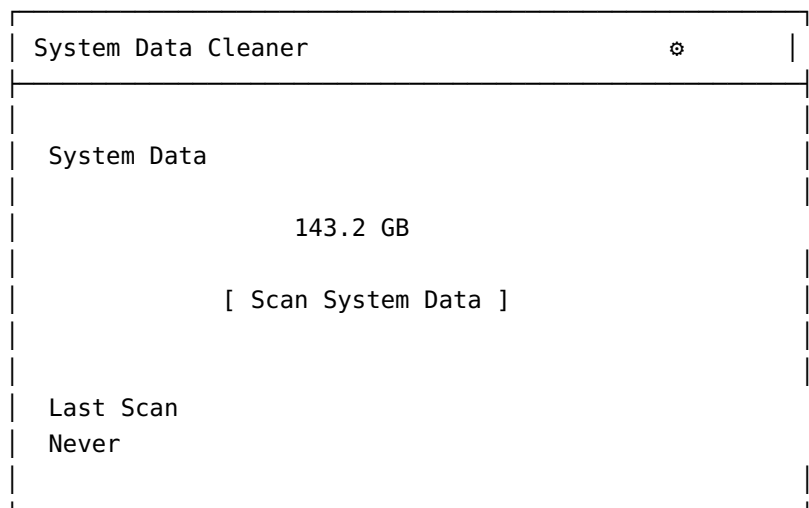
9. User Journey

The intended workflow is:

```
Launch
↓
Scan
↓
Analyze
↓
View large items
↓
Inspect item
↓
Select unwanted item
↓
Review cleanup
↓
Confirm
↓
Move to Trash / delete
↓
Verify
↓
Show recovered storage
```

10. Main UI

The application should have a minimal interface.



The application should not require multiple complex screens before the user can scan.

11. Scan Screen

When scanning:

```
Scanning System Data...  
 76%  
Current location:  
~/Library/Application Support  
Files analyzed:  
1,284,512  
Storage analyzed:  
43.7 GB  
[ Cancel ]
```

The UI must remain responsive.

12. Results Screen

After scanning:

System Data	
143.2 GB	
Sort: Size ▼	
Search: _____	
☐ Docker	24.7 GB
Application Data	Review
☐ Xcode DerivedData	18.4 GB
Developer Data	Safe
☐ Application Caches	9.7 GB
Cache	Safe
☐ iOS Simulator Data	8.6 GB
Developer Data	Review
☐ Logs	4.2 GB
Logs	Safe

Selected:

0 B

[Delete Selected]

13. Large Files View

The application should provide a dedicated large-file view.

Large Files

24.7 GB	Docker.raw
12.4 GB	Project.xcarchive
8.6 GB	Simulator Data
7.2 GB	Ubuntu.qcow2
5.8 GB	macOS Installer

Actions:

Reveal in Finder
Inspect
Move to Trash

14. Directory Analysis

Directories should be expandable.

Example:

~/Library

	Developer	48.6 GB
	Xcode	31.4 GB
	CoreSimulator	17.2 GB
	Application Support	31.7 GB
	Caches	21.4 GB

└─ Logs

8.2 GB

The user should be able to navigate deeper.

15. Item Details

When the user clicks an item:

Xcode DerivedData

18.4 GB

Location:

~/Library/Developer/Xcode/DerivedData

Category:

Developer Data

Type:

Generated Build Data

Files:

83,421

Safety:

LOW RISK

Explanation:

This directory contains generated build artifacts created by Xcode. These files can generally be regenerated by Xcode.

[Reveal in Finder]

[Move to Trash]

16. Terminal Replacement Concept

The application should provide functionality analogous to common Terminal workflows.

Conceptually:

Terminal workflow	Application equivalent
Directory size analysis	Storage Scanner
File search	File Scanner
Sorting	Sort control
Filtering	Search/filter
Path inspection	Details panel
<code>open</code> equivalent	Reveal in Finder
Trash/delete operation	Cleanup engine
Permission errors	Permission UI

The application is not required to execute those commands internally.

It should implement equivalent functionality through native APIs.

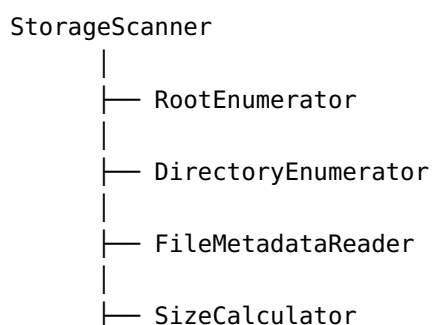
17. Filesystem Scanner

The scanner is the core engineering component.

Responsibilities:

- Enumerate directories.
- Enumerate files.
- Obtain metadata.
- Calculate sizes.
- Handle inaccessible files.
- Handle symbolic links.
- Detect errors.
- Support cancellation.
- Produce structured scan results.

18. Scanner Architecture



19. Scan Locations

The application should not attempt to blindly scan the entire operating system.

Initial scanning should focus on relevant accessible locations.

Examples include:

```
~/Library  
/Library  
/private/var
```

and known application/developer storage locations where appropriate.

The exact list should be configurable and tested against each supported macOS version.

20. User Home Directory

The application should prioritize the user's home directory.

Potentially relevant areas:

```
~/Library/Caches  
~/Library/Logs  
~/Library/Application Support  
~/Library/Developer  
~/Library/Containers  
~/Downloads  
~/Documents
```

However, the application should not assume that everything in these directories is safe to delete.

21. System Locations

System locations require special handling.

Examples:

```
/System  
/Library  
/private/var
```

The application may analyze accessible information but must respect:

- System Integrity Protection
- TCC
- filesystem permissions
- sandbox restrictions
- protected system resources

Protected data should not be forcibly accessed or deleted.

22. Permission Model

Every filesystem operation should first consider whether access is available.

Conceptually:

```
Path  
↓  
Permission Check  
↓  
Accessible?  
├─ YES → Analyze  
└─ NO  → Skip + Report
```

The scanner should not crash when access is denied.

23. Full Disk Access

Some locations may require user-granted Full Disk Access.

If access is required:

```
Some locations cannot be analyzed.  
  
macOS has restricted access to these files.  
  
To analyze additional locations:  
  
System Settings
```

- Privacy & Security
- Full Disk Access

The application should request only permissions that are actually necessary.

24. Sandboxing

The final application architecture must explicitly decide whether it will be:

Sandboxed

Advantages:

- Strong security boundary
- Better alignment with App Store requirements

Disadvantages:

- Broad filesystem scanning is more difficult.
- User-selected filesystem access requires security-scoped permissions.

Non-Sandboxed

Advantages:

- More practical for broad local filesystem analysis.

Disadvantages:

- Greater security responsibility.
- Different distribution requirements.

The project should choose the distribution strategy after validating the required filesystem access.

25. Size Calculation

The scanner should calculate directory sizes recursively.

Conceptually:

```
Directory Size
=
sum(size of files and child directories)
```

However, the application must distinguish between:

Logical Size

The size reported by file metadata.

Physical/Reclaimable Size

The actual disk storage that may be affected by deleting the item.

APFS can make these values different.

26. APFS Considerations

APFS supports mechanisms such as:

- snapshots
- clones
- compression
- sparse files
- shared storage blocks

Therefore the application must avoid misleading statements such as:

"Deleting this 20 GB file will definitely free exactly 20 GB."

Instead:

Logical size:	20.0 GB
Estimated reclaim:	18.7 GB

where an accurate estimate is possible.

Otherwise display:

Reclaimable size cannot be determined precisely.

27. Symbolic Links

The scanner must safely handle symbolic links.

It should not recursively follow arbitrary symlinks by default.

Reasons:

- Infinite loops
- Duplicate traversal

- Unexpected access outside the scan root
- Incorrect size calculations

Recommended initial behavior:

Do not recursively follow symbolic links.

28. Scanner Cancellation

The user must be able to cancel a scan.

Scanning...

[Cancel]

Cancellation should:

- stop new filesystem operations where possible
 - preserve completed results where useful
 - release resources
 - return the UI to a valid state
-

29. Scanner States

Recommended states:

Idle
Preparing
Scanning
Completed
Cancelled
Failed
PermissionRequired

30. Storage Item Model

Conceptual model:

StorageItem

id


```
name
path
isDirectory
size
logicalSize
physicalSize
modifiedDate
category
safetyLevel
isDeletable
```

Not every property needs to be populated for every filesystem object.

31. Category System

The initial category system should remain simple.

```
Cache
Logs
Developer Data
Application Support
Backup
Browser Data
Temporary Data
Virtual Machine
Container Data
Other
Unknown
```

32. Safety Levels

Every cleanup candidate should have a safety classification.

```
SAFE
REVIEW
DANGEROUS
UNKNOWN
```

SAFE

Known regenerable data.

REVIEW

Data that may be removable but could contain user/application information.

DANGEROUS

Data that should not be automatically removed.

UNKNOWN

The application cannot confidently determine the purpose.

Unknown items should not be presented as safe cleanup candidates.

33. Rule Engine

Classification should be implemented using rules.

Example:

```
Rule:
XcodeDerivedDataRule

Path:
~/Library/Developer/Xcode/DerivedData/*

Category:
Developer Data

Safety:
SAFE

Description:
Generated Xcode build artifacts.
```

Another:

```
Rule:
ApplicationCacheRule

Path:
Known application cache locations

Category:
Cache
```

Safety:
SAFE

Rules should be isolated from the scanner.

34. Why Separate Scanner and Classifier?

The scanner answers:

What exists?

The classifier answers:

What is it?

The safety engine answers:

Can it be safely removed?

This separation is important.

```
Filesystem
  ↓
Scanner
  ↓
Storage Items
  ↓
Classifier
  ↓
Category
  ↓
Safety Engine
  ↓
Cleanup Candidate
```

35. Cleanup Engine

The cleanup engine is responsible for deletion.

Architecture:

```
User Selection
  ↓
```

```
Cleanup Plan
  ↓
Path Validation
  ↓
Safety Validation
  ↓
Permission Validation
  ↓
Cleanup Operation
  ↓
Verification
  ↓
Result
```

36. Never Use Unrestricted Deletion

The application should not construct arbitrary shell commands from user input.

Avoid designs such as:

```
rm -rf <user-selected-path>
```

especially when the path originates from UI text or external data.

The cleanup engine should use controlled native filesystem operations.

37. Path Validation

Before any cleanup operation:

1. Confirm item was discovered by scanner.
 2. Confirm path is still valid.
 3. Confirm path is inside an approved scope.
 4. Confirm the item is still the expected item.
 5. Confirm cleanup rule allows removal.
 6. Confirm safety level.
 7. Confirm user selected the item.
 8. Perform operation.
-

38. Path Safety

The application should defend against:

- path traversal
- unexpected symlink changes
- deleted/replaced files
- race conditions
- protected locations
- filesystem changes during scanning

The cleanup engine should never trust a path simply because it appears in the UI.

39. Trash vs Permanent Deletion

Preferred strategy:

```
User selects item
      ↓
Safety validation
      ↓
Move to Trash
```

Moving to Trash provides a recovery mechanism where supported.

Permanent deletion should be restricted to cases where:

- it is technically necessary
 - the user explicitly understands it
 - the operation is clearly identified
-

40. Cleanup Confirmation

Before cleanup:

Review Cleanup

You selected:

Xcode DerivedData	18.4 GB
Application Cache	9.7 GB
Old Logs	4.2 GB

Total:

32.3 GB

Risk:
Low

These files are generally safe to regenerate.

[Cancel]

[Move to Trash]

The user should never be surprised by what is being removed.

41. Cleanup Progress

During cleanup:

Cleaning...

Xcode DerivedData

 91%

18.1 GB / 18.4 GB

[Cancel]

If cancellation occurs, the application should report which operations completed.

42. Cleanup Verification

After cleanup:

Cleanup Complete

Processed:
32.3 GB

Successfully removed:
31.9 GB

Skipped:
0.4 GB

Reason:

Some files changed or became unavailable.

[Done]

The application should not simply assume that the expected amount was removed.

43. Before/After Storage

After cleanup:

Before:
185.0 GB available

After:
216.9 GB available

Recovered:
31.9 GB

The application should explain that storage values can change independently because macOS may perform background storage management.

44. Search

Results should support search.

Example:

Search:
docker

Results:

Docker.raw	24.7 GB
Docker Images	8.2 GB
Docker Volumes	4.1 GB

Search should operate on:

- name
- path
- category

45. Sorting

Users should be able to sort by:

Size

Name

Last Modified

Category

Default:

Largest first.

This matches the application's primary purpose.

46. Filtering

Possible filters:

All

Safe

Review

Large Files

Caches

Developer

Logs

Example:

Filter:

Safe

18.4 GB Xcode DerivedData

9.7 GB Application Cache

4.2 GB Logs

47. Selection System

Users should be able to:

- select one item
- select multiple items
- select all visible safe items
- deselect all
- review selected total

Example:

Selected:
32.3 GB

The selected size should update immediately.

48. "Select All Safe" Feature

A convenient feature:

[Select All Safe]

However, clicking it should **not immediately delete anything**.

It should only select candidates.

Then:

32.3 GB selected
[Review Cleanup]

49. Reveal in Finder

Every filesystem item should optionally support:

Reveal in Finder

This gives the user an external way to inspect the data.

The feature should use the appropriate native macOS mechanism rather than launching arbitrary shell commands.

50. Copy Path

A useful small feature:

```
[ Copy Path ]
```

Example:

```
~/Library/Developer/Xcode/DerivedData
```

This is especially useful for developers.

51. No Mandatory Database

The MVP does not need a database.

The scanner can produce an in-memory model:

```
Scan
↓
[StorageItem]
↓
SwiftUI
```

This reduces:

- application complexity
 - disk usage
 - dependencies
 - maintenance
-

52. Optional Scan History

If storage history is later added, lightweight persistence can be introduced.

For example:

```
ScanDate
TotalStorage
SystemData
CategorySizes
```

This should be added only if needed.

53. Performance Architecture

The scanner may encounter millions of files.

Therefore it should:

- avoid loading every file into memory simultaneously
 - process entries incrementally
 - use asynchronous tasks
 - aggregate sizes efficiently
 - avoid unnecessary metadata requests
 - support cancellation
-

54. Concurrency

Swift Concurrency should be used.

Possible technologies:

```
Task
async/await
TaskGroup
actor
AsyncSequence
```

The UI should run independently from filesystem scanning.

55. Main Thread Rule

Long filesystem operations must not block the main UI thread.

Incorrect:

```
Main Thread
  ↓
Scan millions of files
```

Preferred:

```
Main Actor
  ↓
UI

Background Tasks
  ↓
Filesystem Scanner
```

56. Memory Efficiency

The application should avoid creating a large object graph for every file.

For example, scanning millions of files should not require:

```
1 object per file
+
all objects retained
```

Instead:

```
Enumerate
  ↓
Process
  ↓
Aggregate
  ↓
Discard unnecessary metadata
```

Only items necessary for the UI should be retained.

57. Scan Modes

The MVP can provide:

Quick Scan

Analyze known high-value locations.

Deep Scan

Perform broader analysis.

For example:

Quick Scan

- caches
- logs
- developer data
- known application data

Deep Scan

- broader home directory analysis
- large files
- additional accessible locations

58. Battery Efficiency

The scanner should not run continuously.

Default behavior:

```
Open application
↓
Scan
↓
Results
↓
Cleanup
↓
Close application
```

No daemon should remain running after the application closes.

59. Background Behavior

The application should have:

No background daemon
No continuous scanning
No scheduled scan by default
No filesystem watcher by default

This supports the lightweight product goal.

60. Application Size Requirement

The application should be designed to minimize disk footprint.

Target:

Application:
< 30 MB preferred

No external runtime
No bundled AI model
No bundled database engine
No bundled browser engine
No large assets

The exact size will be measured from the final release build.

61. Memory Target

Target:

Idle:
< 50 MB preferred

During scanning:
Keep memory bounded and proportional to
the amount of UI data retained rather than
the total number of scanned files.

These are engineering targets, not guarantees.

62. CPU Target

When idle:

Approximately 0% CPU

During scanning:

CPU usage should be reasonable and should not unnecessarily monopolize system resources.

63. No Internet Requirement

Core functionality must work offline.

```
Filesystem
  ↓
Local Scanner
  ↓
Local Analysis
  ↓
Local Cleanup
```

No server is required.

64. Privacy

The application should not upload filesystem information.

Never upload by default:

- filenames
- file paths
- directory structures
- document contents
- browser history
- application data

The core application can function completely locally.

65. Analytics

The initial version should preferably have:

No analytics.

If analytics are introduced later, they should be:

- optional
- transparent
- privacy-preserving
- free of filesystem content

66. Security Requirements

Security is critical because the application can delete files.

Requirements:

1. Never bypass macOS security protections.
2. Never disable SIP.
3. Never attempt privilege escalation.
4. Never execute arbitrary shell commands based on paths.
5. Validate every cleanup operation.
6. Respect user permissions.
7. Avoid unnecessary privileges.
8. Never treat unknown data as safe.
9. Never automatically delete critical system files.
10. Protect against filesystem race conditions.

67. Critical System Protection

The application should maintain protected path rules.

Conceptually:

Protected Paths

- └ /System
- └ protected /Library resources
- └ kernel-related resources
- └ security-sensitive resources
- └ other macOS critical locations

The exact protection rules should be implemented conservatively and tested against supported macOS versions.

The app should prefer:

"Cannot delete"

over:

"Probably safe."

68. Unknown Data

Unknown files should appear as:

UNKNOWN

We could not confidently determine
whether this item is safe to remove.

The application should allow inspection but should not encourage deletion.

69. Large Does Not Mean Unnecessary

A fundamental rule:

Large file
≠
Unnecessary file

For example:

Ubuntu.qcow2
20 GB

may be intentionally required by the user.

Therefore the UI should say:

Large file

rather than:

Junk file

unless there is sufficient evidence.

70. Cleanup Recommendation Language

Avoid misleading language.

Bad:

"This file is useless."

Better:

"This appears to be generated cache data and can normally be regenerated."

Bad:

"Delete this to fix your Mac."

Better:

"Removing this data may reclaim storage. The associated application may recreate it."

71. Error Handling

The application must gracefully handle:

- Permission denied
- File disappeared
- Directory disappeared
- File changed
- Disk unavailable
- Operation interrupted
- Unsupported filesystem behavior
- Insufficient permissions
- Protected location

Example:

Some files could not be analyzed.

1,842 items were skipped because macOS
did not provide access.

Your scan has still completed successfully.

72. Race Conditions

The filesystem may change after scanning.

Example:

```
Scan:
file.txt = 10 GB

User waits

Another application deletes file.txt

Cleanup:
file.txt no longer exists
```

The cleanup engine must handle this gracefully.

It should not treat this as a fatal application error.

73. TOCTOU Consideration

The project must account for:

Time-of-check to time-of-use

A path may change between:

```
Check
```

and:

```
Delete
```

Therefore cleanup should verify the target immediately before the operation and use appropriate native filesystem mechanisms.

74. macOS API Strategy

Prefer native APIs.

Primary areas:

```
Foundation
FileManager
URL resource values
File system metadata
NSWorkspace / appropriate Finder integration
FileManager Trash operations
Security-scoped resource handling where required
macOS permission mechanisms
```

The implementation should use documented APIs wherever possible.

75. Terminal Commands

The product is conceptually replacing Terminal commands, but the application should **not depend on Terminal**.

Avoid making the core system:

```
SwiftUI
↓
Process()
↓
shell command
↓
parse stdout
```

because this introduces:

- shell parsing complexity
- security concerns
- localization issues
- command availability differences
- fragile output parsing

Native APIs are preferable.

76. Why Not Just Use `du`?

A command such as `du` can calculate directory sizes, but the application needs more than text output.

The GUI needs structured information:

```
path
size
```

```
category
safety
modified date
cleanup eligibility
```

Native scanning provides structured data directly.

77. Why Not Just Use `rm`?

Because cleanup requires safety validation.

A generic delete command does not understand:

```
Is this generated cache?
Is this a user document?
Is this system data?
Is this path protected?
Did the file change?
Does the user have permission?
```

The application's cleanup engine must make these checks before performing the operation.

78. Recommended Project Structure

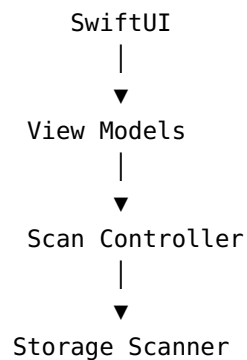
```
SystemDataCleaner/
|
├── App/
│   └── SystemDataCleanerApp.swift
|
├── Models/
│   ├── StorageItem.swift
│   ├── StorageCategory.swift
│   ├── SafetyLevel.swift
│   ├── ScanState.swift
│   └── CleanupOperation.swift
|
├── Views/
│   ├── MainView.swift
│   ├── DashboardView.swift
│   ├── ScanView.swift
│   ├── ResultsView.swift
│   ├── ItemDetailView.swift
│   └── CleanupConfirmationView.swift
|
```

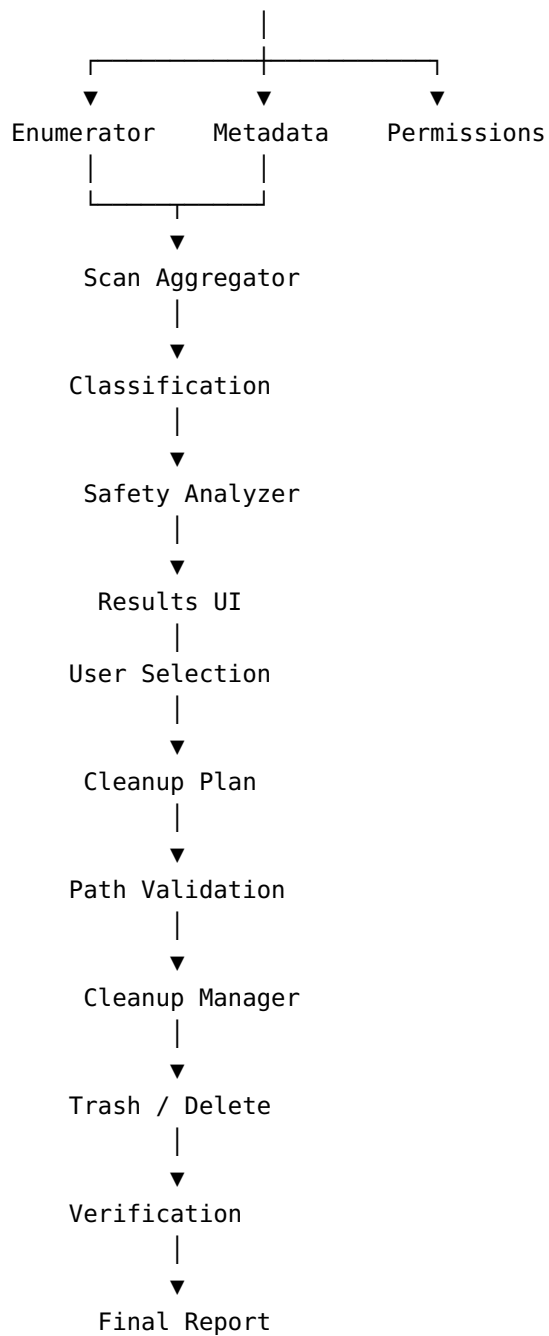
```

├─ ViewModels/
│   ├── DashboardViewModel.swift
│   ├── ScanViewModel.swift
│   └── ResultsViewModel.swift
├─ Scanner/
│   ├── StorageScanner.swift
│   ├── DirectoryScanner.swift
│   ├── FileMetadataReader.swift
│   └── ScanAggregator.swift
├─ Analyzer/
│   ├── CategoryClassifier.swift
│   ├── SafetyAnalyzer.swift
│   └── Rules/
│       ├── CacheRules.swift
│       ├── DeveloperRules.swift
│       ├── LogRules.swift
│       └── ApplicationRules.swift
├─ Cleanup/
│   ├── CleanupManager.swift
│   ├── CleanupPlan.swift
│   ├── PathValidator.swift
│   ├── TrashManager.swift
│   └── CleanupVerifier.swift
├─ Permissions/
│   └── PermissionManager.swift
└─ Tests/
    ├── ScannerTests/
    ├── AnalyzerTests/
    ├── CleanupTests/
    └── SecurityTests/

```

79. Architecture Diagram





80. MVVM Structure

Model

Represents:

- storage items
- categories
- safety levels
- scan results

- cleanup operations

View

Responsible only for presentation.

ViewModel

Responsible for:

- scan state
- selection state
- sorting
- filtering
- UI commands

Filesystem logic should not be placed directly inside SwiftUI views.

81. Scanner Interface

Conceptually:

```
StorageScanner
```

```
scan()  
cancel()
```

It should emit or return:

```
ScanResult
```

The scanner should not know about UI.

82. Classifier Interface

Conceptually:

```
CategoryClassifier
```

```
classify(item)
```

Result:


```
StorageCategory
```

83. Safety Analyzer Interface

Conceptually:

```
SafetyAnalyzer
```

```
evaluate(item)
```

Result:

```
SafetyLevel  
CleanupEligibility  
Reason
```

84. Cleanup Manager Interface

Conceptually:

```
CleanupManager
```

```
createPlan(items)  
validate(plan)  
execute(plan)  
verify(plan)
```

This separates planning from execution.

85. Cleanup Plan

Example:

```
CleanupPlan
```

```
Items:  
- Xcode DerivedData  
- Application Cache
```

- Logs

Estimated size:
32.3 GB

Safety:
LOW

Operation:
Move to Trash

86. Testing Strategy

Testing must be extensive because the application manipulates files.

Unit Tests

Test:

- category rules
- safety rules
- size calculations
- sorting
- filtering
- selection
- path validation

Integration Tests

Test:

- real temporary directories
- scan
- classification
- cleanup
- Trash operations
- permission errors

Security Tests

Test:

- path traversal
- symlinks
- protected paths
- race conditions
- changed files

- inaccessible files
-

87. Test Filesystem

Never test cleanup against the user's real system.

Use:

Temporary Test Root

SystemDataCleanerTests/

- |— cache/
- |— logs/
- |— developer/
- |— large/
- |— unknown/
- |— symlinks/

Tests should create and destroy their own temporary data.

88. Example Unit Test Cases

Test 1

Given:

/cache/example

Expected:

Category = Cache
Safety = Safe

Test 2

Given:

unknown/file.dat

Expected:

Category = Unknown
Safety = Unknown

Test 3

Given protected path:

/System/...

Expected:

Cleanup = Denied

Test 4

Given missing file:

file disappeared

Expected:

Cleanup = Skipped
Application = Continues

89. Performance Testing

Test against:

- 10,000 files
- 100,000 files
- 1,000,000 files
- large directories
- many small files
- large files
- inaccessible directories

Measure:

Scan time
Peak RAM
CPU usage

Cancellation latency
UI responsiveness

90. Performance Benchmark

The project should maintain a benchmark suite.

Example:

```
Benchmark A:  
100,000 files  
  
Benchmark B:  
1,000,000 files  
  
Benchmark C:  
Large developer directory  
  
Benchmark D:  
Large cache directory
```

This prevents future changes from making the application unnecessarily slow.

91. Battery Testing

Measure:

- CPU during Quick Scan
- CPU during Deep Scan
- memory usage
- SSD activity
- scan duration

The application should avoid unnecessary repeated scans.

92. Accessibility

Support:

- VoiceOver
- keyboard navigation
- system appearance
- light/dark mode

- readable text
- accessibility labels

Safety must not depend only on colors.

For example:

```
SAFE
REVIEW
DANGEROUS
```

should appear as text.

93. Native macOS UI

Use:

- SwiftUI
- SF Symbols
- native sheets
- native alerts
- native buttons
- native progress indicators
- system colors

The application should feel like a macOS utility rather than a web application packaged as a desktop app.

94. Distribution

Initial distribution:

```
Build
↓
Code Sign
↓
Hardened Runtime
↓
Notarize
↓
Create DMG
↓
Distribute
```

The final signing and entitlement configuration should be determined based on whether the application is sandboxed.

95. Auto Update

Not required for MVP.

A future version could include an update mechanism, but adding an updater increases:

- application complexity
- attack surface
- maintenance
- distribution size

For the first version, manual updates are acceptable.

96. Logging

Internal diagnostic logging should be lightweight.

Example:

```
Scan started
Root: ~/Library
Files scanned: 1,283,421
Duration: 02:14
Permission errors: 1842
Scan completed
```

Logs must avoid unnecessarily recording sensitive paths or personal information.

97. No Background Daemon

This is an explicit product requirement.

When the user closes the application:

```
SystemDataCleaner
↓
No process
No daemon
```

No watcher
No scanner

unless the user explicitly enables a future feature requiring background functionality.

98. No Network Dependency

Core functions:

Scan
Analyze
Select
Delete
Verify

must work completely offline.

99. No External Runtime

The application must not require users to install:

Python
Node.js
Java
Homebrew
Ruby
Docker

The app should be self-contained.

100. No External CLI Dependency

The scanner should not depend on:

du
find
grep
awk
sed
rm

or other command-line utilities.

Equivalent functionality should be implemented through native APIs.

101. Minimal Dependencies

Preferred dependency strategy:

```
Swift
SwiftUI
Foundation
macOS APIs
```

Third-party packages should only be added when there is a clear engineering benefit.

Every dependency increases:

- application size
 - maintenance
 - security surface
 - build complexity
-

102. MVP Feature Set

The first release should contain only:

Scanning

- Quick Scan
- Directory scanning
- File scanning
- Size calculation
- Permission handling

Analysis

- Large directories
- Large files
- Basic categories
- Safety classification

UI

- Dashboard
- Results list
- Search

- Sort
- Filters
- Details

Cleanup

- Multi-selection
- Cleanup preview
- Safety validation
- Move to Trash
- Cleanup result

Utility

- Reveal in Finder
- Copy path

This is enough for a complete first product.

103. V2 Features

After the MVP is stable:

```
Xcode analyzer
Docker analyzer
Homebrew analyzer
Node cache analyzer
Python cache analyzer
Android/Gradle analyzer
iOS backup analyzer
APFS information
Storage history
```

104. V3 Features

Potential future features:

```
Duplicate detection
Storage trends
Scheduled scans
Menu bar utility
Advanced visualization
CLI
Custom scan locations
```

These should only be added after the core scanner and cleanup engine are highly reliable.

105. Development Phases

Phase 1 — macOS Research

Study:

- FileManager
- URL resource values
- filesystem metadata
- APFS
- permissions
- TCC
- SIP
- sandboxing
- Trash behavior

Deliverable:

Technical feasibility report.

Phase 2 — Scanner Prototype

Implement:

```
DirectoryScanner  
FileMetadataReader  
SizeCalculator
```

No deletion.

Deliverable:

CLI/debug output showing largest directories and files.

Phase 3 — Classification

Implement:

CategoryClassifier
SafetyAnalyzer
Rules

Deliverable:

Structured storage analysis.

Phase 4 — SwiftUI Interface

Implement:

- Dashboard
- Scan screen
- Results screen
- Details screen

Deliverable:

Fully functional read-only analyzer.

Phase 5 — Cleanup Engine

Implement:

CleanupPlan
PathValidator
SafetyValidator
TrashManager
CleanupVerifier

Deliverable:

Safe cleanup prototype.

Phase 6 — Security Testing

Perform:

- path traversal tests
- symlink tests
- protected path tests
- permission tests
- race-condition tests

Deliverable:

Security-tested cleanup engine.

Phase 7 — Performance Optimization

Test:

- large directories
- millions of files
- memory
- CPU
- cancellation

Deliverable:

Production-ready scanner.

Phase 8 — Packaging

Implement:

- release build
- code signing
- hardened runtime
- notarization
- DMG

Deliverable:

Installable macOS application.

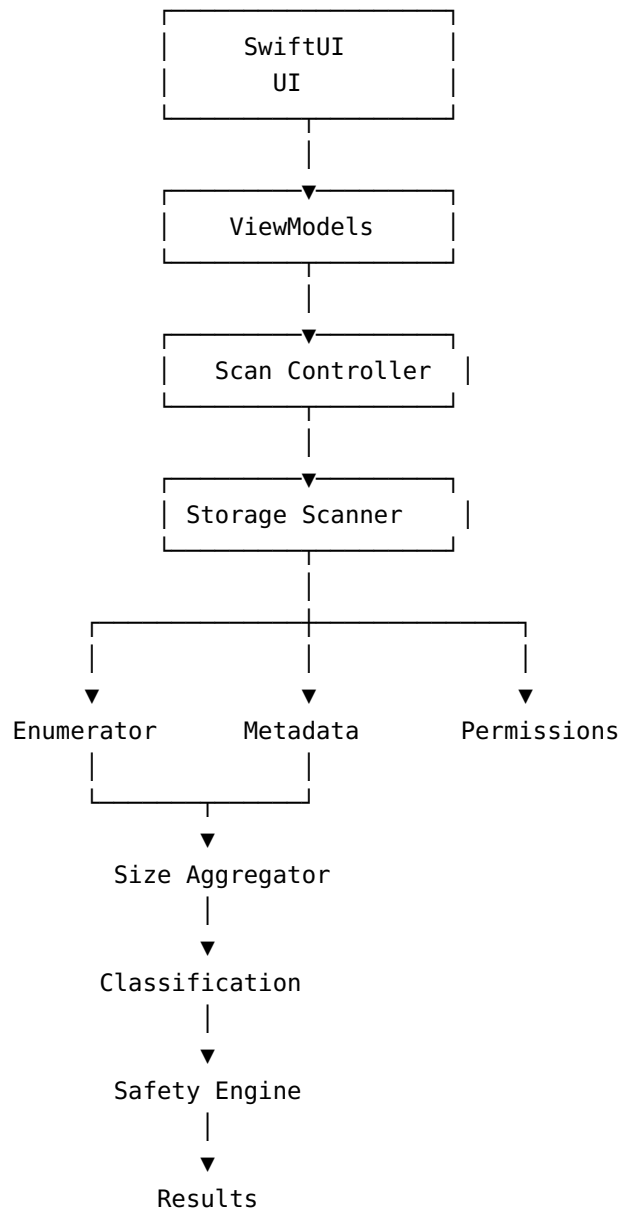
106. Definition of Done

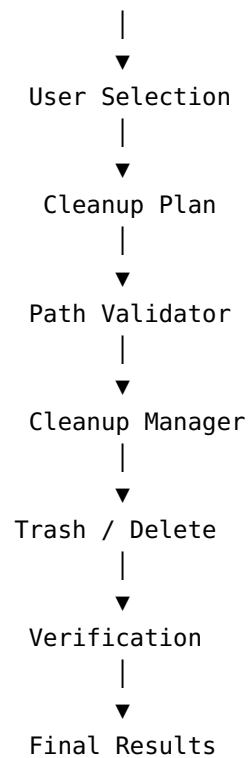
The MVP is complete when:

- The application launches without internet access.
- The application is lightweight.
- Users can start a scan with one action.
- The scanner finds large accessible files/directories.
- Results are sorted by size.
- Users can search results.
- Users can inspect paths.
- Users can reveal files in Finder.
- Known cleanup candidates are classified.
- Unknown data is not presented as safe.

- Users explicitly select cleanup items.
- Every cleanup operation is validated.
- Protected locations cannot be accidentally removed.
- Permission failures are handled.
- Cleanup results are verified.
- The UI remains responsive during scanning.
- The application does not require Terminal.
- The application does not require Python/Node/etc.
- The application does not require a backend.
- Core functionality works offline.
- Unit, integration, and security tests pass.

107. Final Technical Architecture





108. Final Product Definition

Product

System Data Cleaner

One-sentence description

A lightweight native macOS application that replaces Terminal-based storage investigation with a user-friendly interface for finding, inspecting, selecting, and safely removing unnecessary files and directories.

Core workflow

```
SCAN
↓
FIND LARGE ITEMS
↓
INSPECT
↓
SELECT
↓
REVIEW
↓
```

```
VALIDATE
↓
TRASH / DELETE
↓
VERIFY
```

Core technology

```
Swift
+
SwiftUI
+
Foundation
+
macOS APIs
```

Explicitly excluded

```
AI
Cloud
Backend
Electron
Python
Node.js
External runtimes
Background daemon
Mandatory internet
```

Product philosophy

Find it. Understand it. Choose it. Clean it.

The application should not attempt to be the biggest or most feature-heavy Mac cleaner. Its advantage should be that it is **small, native, transparent, focused, and safe**.

The central problem it solves is simple:

"My Mac says System Data is huge. Show me what's actually taking the space, and let me decide what to remove."